

Problem 01: FIFO algorithm.

Source Code:

```
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;
#ifdef ONLINE_JUDGE
#define debug(x) cerr << #x << " = " << x << endl
#else
#define debug(x)
#endif

using ll = long long;

void Solve()
{
    ll n, f;
    cin >> n >> f;
    string s;
    cin >> s;

    queue<char> q;
    unordered_set<char> st;

    ll hit = 0, fault = 0;

    for (char c : s)
    {
        if (st.count(c))
        {
            hit++;
        }
        else
        {
            fault++;
            if ((ll)q.size() == f)
            {
                char rem = q.front();
                q.pop();
                st.erase(rem);
            }
            q.push(c);
            st.insert(c);
        }
    }
}
```

```

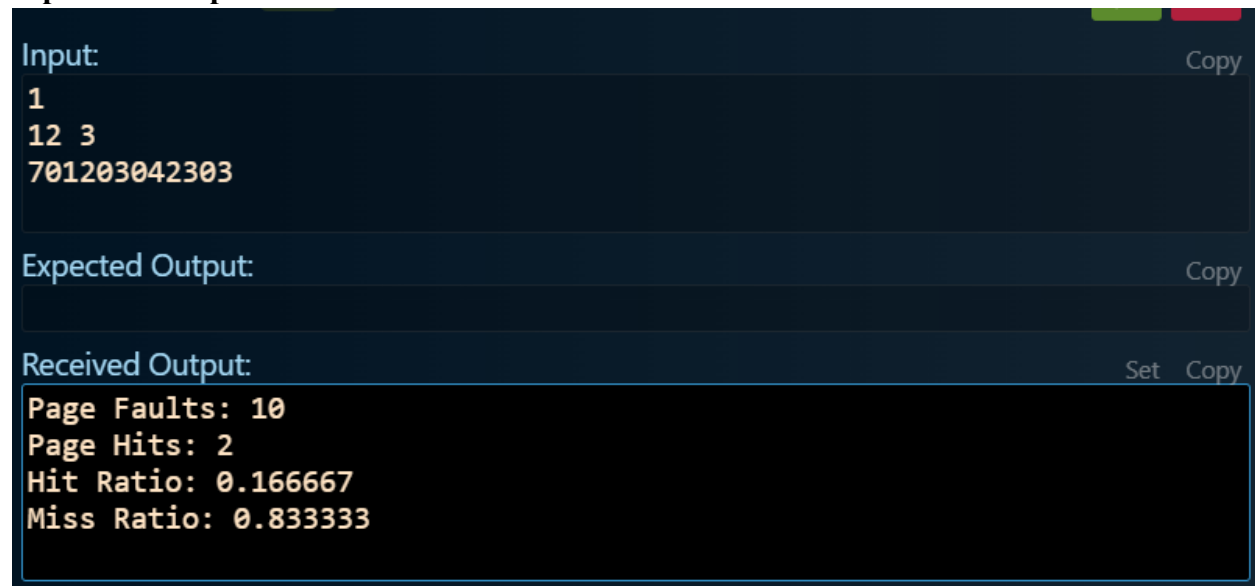
double hit_ratio = (double)hit / n;
double miss_ratio = (double)fault / n;

cout << "Page Faults: " << fault << "\n";
cout << "Page Hits: " << hit << "\n";
cout << fixed << setprecision(6);
cout << "Hit Ratio: " << hit_ratio << "\n";
cout << "Miss Ratio: " << miss_ratio << "\n";
}

int32_t main()
{
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    ll t = 1;
    cin >> t;
    while (t--)
        Solve();
    return 0;
}

```

Input and Output :



The screenshot shows a code editor with three sections: Input, Expected Output, and Received Output. The Input section contains the values 1, 12 3, and 701203042303. The Expected Output section is empty. The Received Output section displays the results of the FIFO algorithm: Page Faults: 10, Page Hits: 2, Hit Ratio: 0.166667, and Miss Ratio: 0.833333.

```

Input:
1
12 3
701203042303

Expected Output:

Received Output:
Page Faults: 10
Page Hits: 2
Hit Ratio: 0.166667
Miss Ratio: 0.833333

```

Figure 6.1: Input and Output of FIFO algorithm.

Problem 02: LRU algorithm.

Source Code:

```
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;
#ifdef ONLINE_JUDGE
#define debug(x) cerr << #x << " = " << x << endl
#else
#define debug(x)
#endif

using ll = long long;

void Solve()
{
    ll n, f;
    cin >> n >> f;
    string s;
    cin >> s;

    list<char> l;
    unordered_map<char, list<char>::iterator> mp;

    ll hit = 0, fault = 0;

    for (char c : s)
    {
        if (mp.count(c))
        {
            hit++;
            l.erase(mp[c]);
            l.push_front(c);
            mp[c] = l.begin();
        }
        else
        {
            fault++;
            if ((ll)l.size() == f)
            {
                char rem = l.back();
                l.pop_back();
                mp.erase(rem);
            }
            l.push_front(c);
        }
    }
}
```

```

        mp[c] = l.begin();
    }
}

double hit_ratio = (double)hit / n;
double miss_ratio = (double)fault / n;

cout << "Page Faults: " << fault << "\n";
cout << "Page Hits: " << hit << "\n";
cout << fixed << setprecision(6);
cout << "Hit Ratio: " << hit_ratio << "\n";
cout << "Miss Ratio: " << miss_ratio << "\n";
}

int32_t main()
{
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    ll t = 1;
    cin >> t;
    while (t--)
        Solve();
    return 0;
}

```

Input and Output:

Input:	Copy
1 12 3 701203042303	
Expected Output:	Copy
Received Output:	Set Copy
Page Faults: 9 Page Hits: 3 Hit Ratio: 0.250000 Miss Ratio: 0.750000	

Figure 6.2: Input and Output of LRU algorithm.